



Kaunas University of Technology

Faculty of Informatics

# Lab1

---

**Yoga Darsa Vijayakumar Maheswari**

Project author

**prof. dr. Dalia Čalnerytė**  
**assoc. prof. dr. Andrius Kriščiūnas**

Professors

---

**Kaunas, 2026**

## Contents

---

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>1. Introduction .....</b>                                                | <b>3</b>  |
| <b>2. Task 1: Fruit Image Generation Using an AC-GAN .....</b>              | <b>3</b>  |
| 2.1 Dataset .....                                                           | 3         |
| 2.2 AC-GAN Architecture .....                                               | 4         |
| 2.3 GAN Training Setup .....                                                | 4         |
| 2.4 GAN Performance Evaluation .....                                        | 5         |
| 2.5 Classification Model (SimpleCNN) .....                                  | 6         |
| 2.6 Real vs. Synthetic Data Comparison .....                                | 6         |
| 2.7 Effect of Training-Data Amount .....                                    | 8         |
| 2.8 Task 1 Findings .....                                                   | 8         |
| <b>3. Task 2: Image-to-Image Translation (Pix2Pix &amp; CycleGAN) .....</b> | <b>9</b>  |
| 3.1 Dataset .....                                                           | 9         |
| 3.2 Pix2Pix (Conditional GAN, supervised) .....                             | 9         |
| 3.3 CycleGAN (unpaired translation, unsupervised) .....                     | 9         |
| 3.4 Training Setup .....                                                    | 10        |
| 3.5 Evaluation .....                                                        | 10        |
| 3.6 Results Summary .....                                                   | 11        |
| <b>5. Conclusion .....</b>                                                  | <b>12</b> |

# 1. Introduction

This report explores deep-learning approaches to generative modelling and image classification using Generative Adversarial Networks (GANs) and Convolutional Neural Networks (CNNs). Two tasks are addressed:

- **Task 1** — an Auxiliary Classifier GAN (AC-GAN) is implemented from scratch to synthesise fruit images, its generation quality is evaluated, and a custom CNN classifier is trained on real versus synthetic data while the effect of dataset size on accuracy and computational cost is analysed.
- **Task 2** — two image-to-image translation frameworks are compared on a satellite–map dataset: a supervised conditional GAN (Pix2Pix) and an unsupervised CycleGAN, evaluated with PSNR, SSIM and FID alongside training time and model size.

All models were implemented in the PyTorch ecosystem with GPU (CUDA) acceleration. No transfer learning or pretrained weights were used; every network was trained from random initialisation, as required by the assignment.

## 2. Task 1: Fruit Image Generation Using an AC-GAN

### 2.1 Dataset

The dataset contains three balanced fruit classes — Apple, Banana and Orange — organised into separate train, validation and test folders and loaded with torchvision’s ImageFolder. Every image is resized to 64×64 and normalised to the  $[-1, 1]$  range, which matches the Tanh output of the generator and stabilises GAN training.

| Split      | Apple | Banana | Orange | Total |
|------------|-------|--------|--------|-------|
| Train      | 70    | 70     | 70     | 210   |
| Validation | 20    | 20     | 20     | 60    |
| Test       | 10    | 10     | 10     | 30    |

Table 1 — Dataset composition. Classes are perfectly balanced (70/20/10 per class).

**Explanation** — the table lists how many images of each fruit are available in each split. Because every class has the same count, the classifier cannot gain accuracy simply by favouring a majority class — any measured accuracy reflects genuine discrimination. The headline figure is the small training total (210 images), which is the main bottleneck for both GAN realism and classifier performance later in the report.

The dataset is small (210 training images), which is a key constraint that later limits both the realism of the generated images and the accuracy of the synthetic-data classifier. Pre-processing:

```
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)), # -> [-1, 1]
])
```

- **Preprocessing:** Every image was resized to a strict 64×64 pixel resolution. Pixel values were normalized to a  $[-1, 1]$  tensor range to match the Generator's Tanh output.

## 2.2 AC-GAN Architecture

An Auxiliary Classifier GAN (AC-GAN) was built entirely from scratch. Unlike a plain DCGAN, the AC-GAN conditions generation on a class label and forces the discriminator to also predict that label, which pushes the generator toward class-distinguishable output.

### Generator

The generator receives a 100-dimensional noise vector  $z$  and a class label. The label is mapped through an embedding layer to a 100-dimensional vector and combined with the noise by element-wise multiplication (conditioning), then upsampled to a  $3 \times 64 \times 64$  RGB image through five transposed-convolution blocks with BatchNorm and ReLU; the final layer uses Tanh.

```
self.label_emb = nn.Embedding(num_classes, nz) # nz = 100
self.main = nn.Sequential(
    nn.ConvTranspose2d(nz, 512, 4, 1, 0, bias=False), nn.BatchNorm2d(512), nn.ReLU(True),
    nn.ConvTranspose2d(512, 256, 4, 2, 1, bias=False), nn.BatchNorm2d(256), nn.ReLU(True),
    nn.ConvTranspose2d(256, 128, 4, 2, 1, bias=False), nn.BatchNorm2d(128), nn.ReLU(True),
    nn.ConvTranspose2d(128, 64, 4, 2, 1, bias=False), nn.BatchNorm2d(64), nn.ReLU(True),
    nn.ConvTranspose2d(64, 3, 4, 2, 1, bias=False), nn.Tanh()) # -> 3 x 64 x 64

def forward(self, noise, labels):
    emb = self.label_emb(labels).view(-1, nz, 1, 1)
    return self.main(noise * emb) # element-wise conditioning
```

### Discriminator

The discriminator is a strided-convolution stack ( $3 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$  channels, LeakyReLU(0.2), BatchNorm) feeding two heads: an adversarial head with Sigmoid (real vs. fake) and an auxiliary head with Softmax over the three classes. This dual objective is what distinguishes AC-GAN from an ordinary GAN.

| Component     | Input                              | Output                                                  |
|---------------|------------------------------------|---------------------------------------------------------|
| Generator     | noise $z$ (100-d), class embedding | RGB image $3 \times 64 \times 64$ , values in $[-1, 1]$ |
| Discriminator | RGB image $3 \times 64 \times 64$  | (1) real/fake probability (2) class logits (3-way)      |

Table 2 — AC-GAN inputs and outputs.

**Explanation** — the table summarises what each network reads and produces. The generator takes pure noise plus a class label and outputs an image, so it is responsible for creativity; the discriminator reads an image and outputs two things at once — a real/fake score and a class prediction. The second (auxiliary) output is the defining feature of an AC-GAN: forcing the discriminator to name the fruit pushes the generator to make each class visually distinct.

## 2.3 GAN Training Setup

The AC-GAN was trained adversarially for 100 epochs. The generator loss equally weights the adversarial term and the auxiliary classification term; the discriminator is trained on both real and (detached) fake batches.

| Hyper-parameter | Value                                        |
|-----------------|----------------------------------------------|
| Epochs          | 100                                          |
| Optimizer       | Adam ( $\beta_1 = 0.5$ , $\beta_2 = 0.999$ ) |
| Learning rate   | 0.0002                                       |
| Batch size      | 32                                           |
| Image size      | $64 \times 64$ RGB                           |

| Hyper-parameter       | Value                                                       |
|-----------------------|-------------------------------------------------------------|
| Latent dimension (nz) | 100                                                         |
| Adversarial loss      | BCELoss                                                     |
| Auxiliary loss        | CrossEntropyLoss                                            |
| Generator loss        | $0.5 \cdot \text{adversarial} + 0.5 \cdot \text{auxiliary}$ |
| GAN training time     | $\approx 48.5$ s (GPU)                                      |

Table 3 — AC-GAN training configuration.

**Explanation** — these are the settings used to train the GAN. The values are the standard DCGAN recipe: Adam with  $\beta_1 = 0.5$  (rather than the usual 0.9) and a learning rate of 0.0002 are the well-known defaults that keep adversarial training stable. The 0.5 / 0.5 split in the generator loss means realism (fooling the discriminator) and class-correctness are weighted equally. The whole GAN trained in about 48 seconds on a GPU, which is fast because the dataset and image size are small.

## 2.4 GAN Performance Evaluation

Generator quality was assessed both visually (per-class grids of generated images, see the generated samples) and quantitatively. The training curves below show the generator and discriminator losses settling after the early adversarial swings, while the discriminator’s classification accuracy on real images rapidly climbs toward 100% — evidence that the auxiliary head learns the class structure.

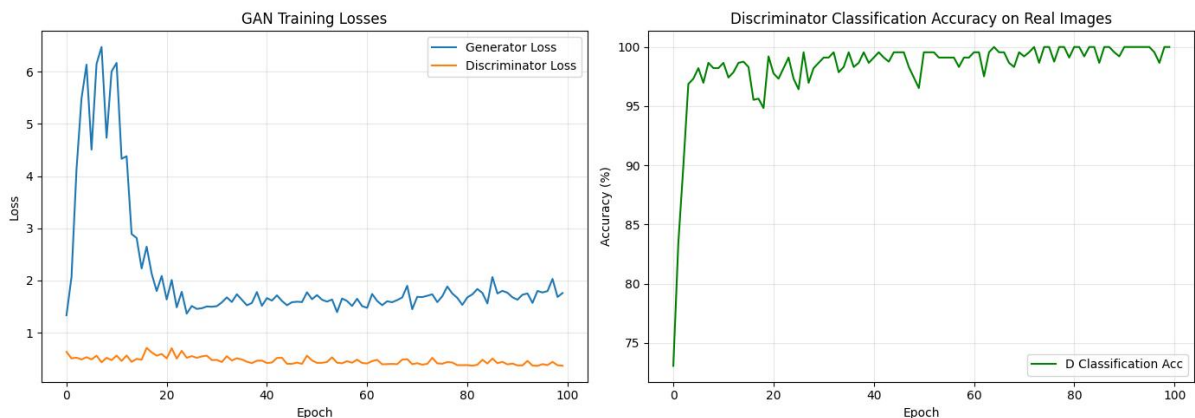


Figure 1 — AC-GAN training: generator/discriminator loss (left) and discriminator class-accuracy on real images (right).

**Explanation** — the left plot shows the adversarial tug-of-war: the generator loss spikes in the first few epochs while it is still learning, then settles into a stable band, and the discriminator loss stays low and steady. Crucially, neither line diverges or flatlines to zero, which would signal training collapse — so the training was healthy. The right plot shows the discriminator’s class accuracy on real images climbing to nearly 100% within a few epochs, confirming the auxiliary head quickly learned to tell the three fruits apart.

**Quantitative generation metric.** To measure how class-distinguishable the generated images are, 500 images were synthesised with random labels and passed through the trained discriminator’s auxiliary head. The discriminator recovered the intended class **85.6%** of the time — far above the 33.3% random baseline — confirming that class conditioning works.

| Class   | Generation class-accuracy | Correct / total |
|---------|---------------------------|-----------------|
| Apple   | 98.0%                     | 146 / 149       |
| Banana  | 73.4%                     | 138 / 188       |
| Orange  | 88.3%                     | 144 / 163       |
| Overall | 85.6%                     | (500 samples)   |

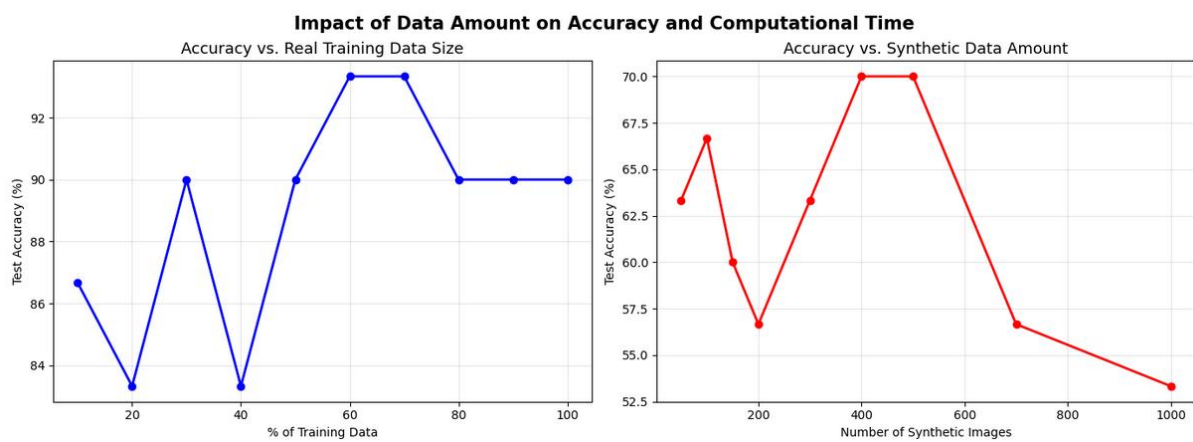
Table 4 — Per-class accuracy of the discriminator on AC-GAN-generated images. Banana is the hardest class to generate distinctly.

**Explanation** — this measures how recognisable the generated images are. 500 images were created with known labels and fed back to the discriminator; if the generated “Apple” really looks like an apple, the discriminator should label it Apple. An overall 85.6% (against a 33.3% random baseline) means the generator is genuinely producing class-specific images, not random noise. Banana scores lowest (73.4%), which tells us the generator finds bananas the hardest to render distinctly — a weakness that reappears in the classification results below.

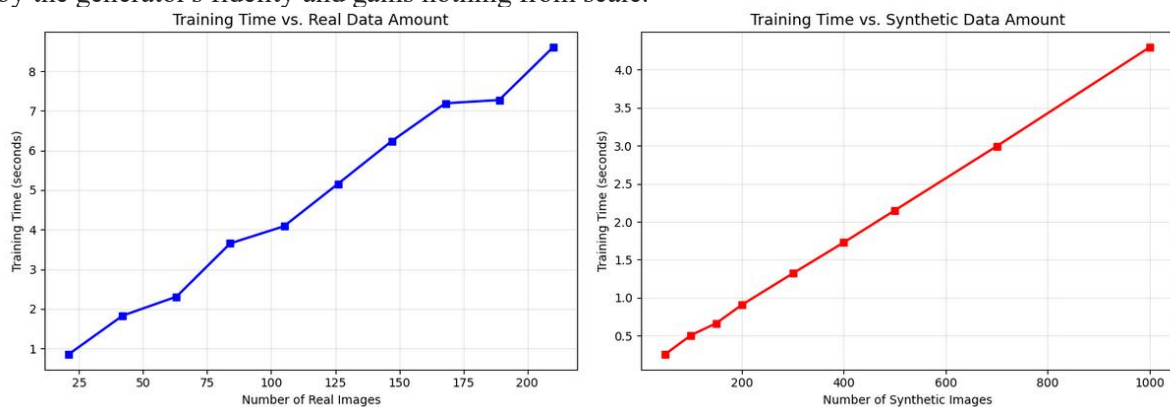
## 2.5 Classification Model (SimpleCNN)

A custom SimpleCNN (no transfer learning) is used for all classification experiments so comparisons are fair. It has three convolutional blocks (32 → 64 → 128 channels, each Conv–ReLU–MaxPool) followed by a fully-connected head with dropout.

## 2.6 Real vs. Synthetic Data Comparison



**Explanation** — the two plots visualise the same numbers as Tables 8 and 9 and make the contrast immediate. The blue curve climbs from about 87% at the smallest slice of real data to a peak near 93% around 60–70% of the dataset, then plateaus. The visible wobbles (the dips at 20% and 40%) are not a model problem — they come from the tiny 30-image test set, where every single misclassification is worth ~3.3%, so one bad prediction visibly moves the line. The red curve tells a very different story: synthetic-data accuracy is trapped in the 53–70% band no matter how many images are added. It briefly peaks at 400–500 generated images and then drifts downward, ending lower at 1000 images (53.3%) than at 50 (63.3%). The shape captures the central finding of Task 1 — real data has a higher ceiling and rewards more samples up to a plateau, while synthetic data is capped by the generator's fidelity and gains nothing from scale.



**Explanation** — both curves are essentially straight lines, confirming that training cost scales linearly with the number of images — exactly what is expected when the optimiser runs for a fixed number of epochs over an increasing dataset. The slopes, however, are very different: real-data training costs roughly 0.04 s per image (210 images → ~8.6 s), whereas synthetic training costs about 0.004 s per

image (1000 images  $\rightarrow$   $\sim$ 4.3 s) — an order of magnitude faster. The reason is purely an I/O effect: the AC-GAN-generated tensors already live in GPU memory, so each epoch skips reading from disk, decoding JPEGs and copying data from CPU to GPU. The synthetic plot is almost perfectly straight because there is no I/O variability; the real-data plot shows small kinks from filesystem and decoder jitter. Read together with Figure 5, the picture is clear: synthetic training is cheaper but does not buy more accuracy beyond the generator's ceiling.

The SimpleCNN was trained separately on (a) the real training set and (b) 500 AC-GAN-generated images, then both were evaluated on the same real test set.

| Training data          | Test accuracy | Training time |
|------------------------|---------------|---------------|
| Real (210 images)      | 93.33%        | 9.26 s        |
| Synthetic (500 images) | 60.00%        | 2.17 s        |
| Gap                    | 33.3 pts      | —             |

Table 5 — Classifier trained on real vs. synthetic data, evaluated on the real test set.

**Explanation** — both classifiers are tested on the same real images, so the comparison is fair. Real data wins decisively on accuracy (93.3% vs. 60.0%) because real images carry richer, sharper, more diverse detail. Synthetic training is roughly four times faster because the generated tensors already sit in memory, avoiding disk reading and image decoding — but that speed comes at a heavy accuracy cost. The 33-point gap is the key takeaway: synthetic images alone are not a substitute for real data here.

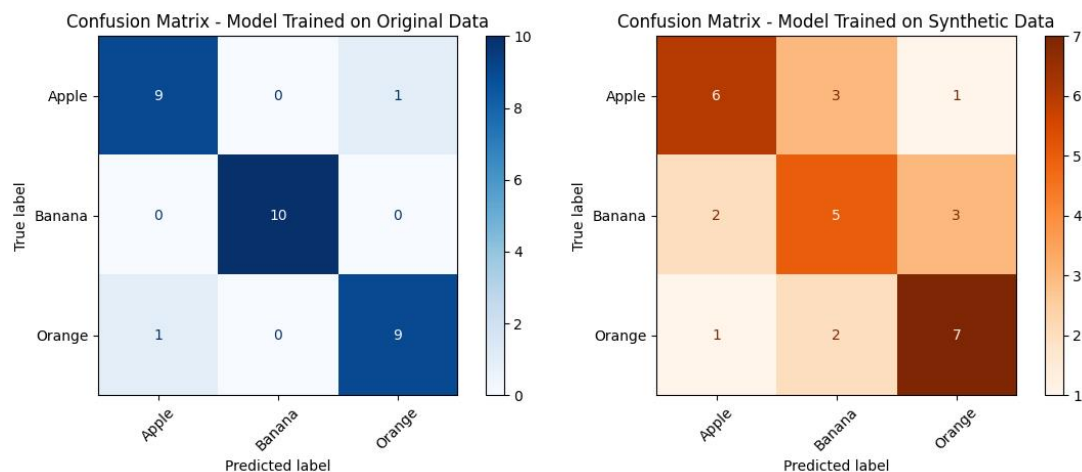


Figure 2 — Confusion matrices: real-data model (left, blue) and synthetic-data model (right, orange).

**Explanation** — in a confusion matrix the rows are the true fruit and the columns are the predicted fruit, so correct predictions sit on the diagonal and mistakes sit off it. The real-data matrix (left) is almost entirely diagonal — only a couple of stray errors. The synthetic-data matrix (right) has many off-diagonal cells, with Banana being mixed up most often, giving a visual confirmation of the weak scores in Table 7.

## 2.7 Effect of Training-Data Amount

The classifier was retrained (30 epochs each) on increasing fractions of real data and increasing counts of synthetic data, recording both accuracy and training time.

| Real images | 21   | 42   | 63   | 84   | 105  | 126  | 147  | 168  | 189  | 210  |
|-------------|------|------|------|------|------|------|------|------|------|------|
| Acc (%)     | 86.7 | 83.3 | 90.0 | 83.3 | 90.0 | 93.3 | 93.3 | 90.0 | 90.0 | 90.0 |
| Time (s)    | 0.86 | 1.83 | 2.31 | 3.65 | 4.09 | 5.15 | 6.23 | 7.19 | 7.27 | 8.60 |

Table 8 — Real-data scaling. Accuracy rises then plateaus (~90–93%); time grows roughly linearly with data.

**Explanation** — each column retrains the classifier on a larger slice of real data. Accuracy climbs from ~87% at 21 images to a plateau around 90–93% — once there are enough examples, extra data adds little. The small wobbles (e.g. the dip at 84 images) come from the tiny 30-image test set, where each image is worth 3.3%, so one misclassification visibly moves the number. Training time grows almost in step with the number of images, as expected.

| Synthetic images | 50   | 100  | 150  | 200  | 300  | 400  | 500  | 700  | 1000 |
|------------------|------|------|------|------|------|------|------|------|------|
| Acc (%)          | 63.3 | 66.7 | 60.0 | 56.7 | 63.3 | 70.0 | 70.0 | 56.7 | 53.3 |
| Time (s)         | 0.26 | 0.51 | 0.66 | 0.91 | 1.32 | 1.73 | 2.15 | 2.99 | 4.29 |

Table 9 — Synthetic-data scaling. Accuracy stays in the 53–70% band — more synthetic data does not help, because quality is capped by the generator.

**Explanation** — unlike real data, pouring in more synthetic images does not raise accuracy — it hovers in the 53–70% band and even drifts down at 1000 images. The reason is that every synthetic image is drawn from the same imperfect generator, so extra samples repeat the same flaws rather than adding new information; beyond a point they act more like noise. Training time still scales linearly. Together, Tables 8 and 9 show the central lesson: real data has a higher ceiling that rewards more samples, whereas synthetic data is capped by the generator’s fidelity.

## 2.8 Task 1 Findings

- The AC-GAN, trained entirely from scratch, generated recognisably class-conditioned fruit images; the discriminator classified them with 85.6% accuracy (vs. 33.3% chance).
- Banana was consistently the hardest class for both generation (73.4%) and synthetic-data classification ( $F1 = 0.50$ ).
- A classifier on real data reached 93.3% accuracy versus 60.0% on synthetic data — a 33-point gap — confirming that synthetic data alone is insufficient here.
- Real-data accuracy improves with dataset size then plateaus; synthetic-data accuracy is capped by generator quality. Training time scales roughly linearly in both cases, and synthetic training is faster because data is already tensorised in memory.

## 3. Task 2: Image-to-Image Translation (Pix2Pix & CycleGAN)

Task 2 translates satellite imagery into map imagery using two paradigms: the supervised Pix2Pix and the unsupervised CycleGAN. Both were trained on the same maps dataset and compared with PSNR, SSIM and FID, plus training time and model size.

### 3.1 Dataset

Each source file is a  $1200 \times 600$  concatenation of a satellite view (left) and the matching map view (right). A custom Dataset splits every image down the middle, giving a  $600 \times 600$  satellite input (domain A) and a  $600 \times 600$  map target (domain B); both are resized to  $256 \times 256$  and normalised to  $[-1, 1]$ .

| Property            | Value                                                 |
|---------------------|-------------------------------------------------------|
| Training images     | 1096                                                  |
| Validation images   | 1098                                                  |
| Raw dimensions      | $1200 \times 600$ (paired)                            |
| Satellite input (A) | $600 \times 600 \rightarrow$ resized $256 \times 256$ |
| Map target (B)      | $600 \times 600 \rightarrow$ resized $256 \times 256$ |
| Normalisation       | $[-1, 1]$                                             |

Table 10 — Satellite-map dataset.

**Explanation** — the table records the dataset used for translation. Each source file glues a satellite photo and its matching map side by side, so the loader slices it down the middle to obtain an input (satellite) and a target (map). Because the two halves come from the exact same location, the data is “paired” — which is what makes the supervised Pix2Pix approach possible. Everything is resized to  $256 \times 256$  and scaled to  $[-1, 1]$  to match the network outputs.

### 3.2 Pix2Pix (Conditional GAN, supervised)

Pix2Pix uses a U-Net generator with skip connections (8 down-sampling + 7 up-sampling blocks with InstanceNorm and dropout) so spatial structure is preserved between encoder and decoder. The discriminator is a PatchGAN that judges  $16 \times 16$  local patches rather than the whole image, focusing learning on local texture; it sees the satellite and map concatenated (6 input channels).

**Loss.** The generator combines an adversarial loss with an L1 reconstruction loss (weight  $\lambda = 100$ ). Importantly, the executed code uses **MSELoss (least-squares GAN)** for the adversarial term — not BCE — which is more stable for image translation:

### 3.3 CycleGAN (unpaired translation, unsupervised)

CycleGAN removes the need for paired data. It uses two generators —  $G_{AB}$  (satellite $\rightarrow$ map) and  $G_{BA}$  (map $\rightarrow$ satellite) — and two PatchGAN discriminators ( $D_A$ ,  $D_B$ ). A cycle-consistency loss enforces that translating an image to the other domain and back returns (approximately) the original.

**Loss.** Adversarial (MSELoss) terms for both directions plus a cycle-consistency loss (L1, weight 10):

### 3.4 Training Setup

| Setting          | Pix2Pix                       | CycleGAN                        |
|------------------|-------------------------------|---------------------------------|
| Type             | Supervised (paired)           | Unsupervised (unpaired)         |
| Epochs           | 10                            | 5                               |
| Networks         | 1 generator + 1 discriminator | 2 generators + 2 discriminators |
| Optimizer        | Adam (0.5, 0.999), lr 2e-4    | Adam (0.5, 0.999), lr 2e-4      |
| Batch size       | 4                             | 4                               |
| Adversarial loss | MSE (LSGAN)                   | MSE (LSGAN)                     |
| Extra loss       | $L1 \times 100$               | cycle $L1 \times 10$            |

Table 11 — Training configuration for both models.

**Explanation** — placing the two setups side by side highlights why they behave differently. Pix2Pix trains a single generator–discriminator pair on paired data and adds a strong L1 term ( $\times 100$ ) that forces its output to match the known target pixel-for-pixel. CycleGAN trains four networks and, lacking paired targets, relies on a cycle-consistency term ( $\times 10$ ) — translate to the other domain and back, and you should recover the original. Both use the more stable least-squares (MSE) adversarial loss.

### 3.5 Evaluation

Models were evaluated on the validation set with three metrics: PSNR and SSIM (higher = closer to ground truth) and FID (lower = closer generated/real distributions). A side-by-side comparison of a representative sample is shown below.

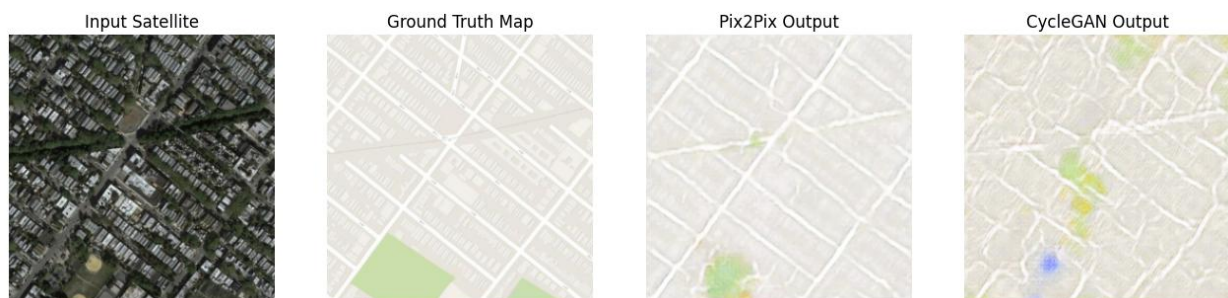


Figure 3 — Input satellite, ground-truth map, Pix2Pix output and CycleGAN output for one validation sample. Pix2Pix reproduces road structure more faithfully; CycleGAN is blurrier with colour artefacts.

**Explanation** — reading left to right: the first panel is the satellite input, the second is the true map the model should produce, and the last two are what each model actually generated. Pix2Pix’s output keeps the road grid sharp and close to the ground truth, thanks to its pixel-matching L1 loss. CycleGAN’s output is recognisable but softer, with washed-out streets and stray colour blotches — the visible price of learning without paired targets.

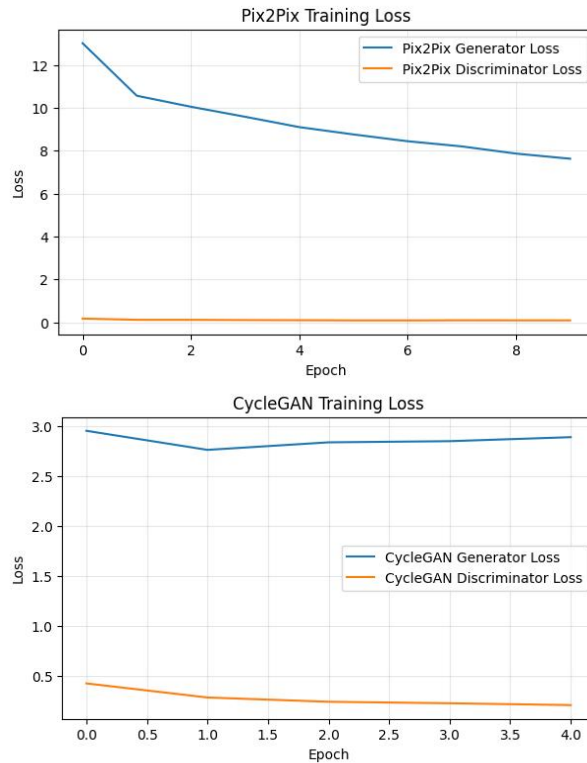


Figure 4 — Training loss curves: Pix2Pix (left) and CycleGAN (right).

**Explanation** — both plots show losses decreasing as training progresses, indicating stable learning. Pix2Pix’s generator loss starts high — because the L1 term is multiplied by 100 — and falls steadily as the output gets closer to the target, while its discriminator loss stays low. CycleGAN’s generator loss is a composite of two adversarial terms plus the cycle loss, so it sits at a different scale; its smoothly declining discriminator loss shows the two domains learning to separate. Neither model shows the wild oscillation that would indicate instability.

### 3.6 Results Summary

| Model    | Type         | Train time | Generator size | Total size train | SSIM<br>↑ | PSNR<br>↑ | FID<br>↓* |
|----------|--------------|------------|----------------|------------------|-----------|-----------|-----------|
| Pix2Pix  | Supervised   | 11.08 min  | 207.5 MB       | 218 MB (G+D)     | 0.782     | 26.53     | 0.13      |
| CycleGAN | Unsupervised | 16.23 min  | 207.5 MB       | 436 MB (2G+2D)   | 0.583     | 20.70     | 0.32      |

Table 12 — Final comparison (dataset-level averages over 100 validation batches). \*Fallback pixel-feature FID — ranking only. Generator (inference) size is identical because both reuse the same U-Net; CycleGAN’s larger total reflects four networks trained simultaneously.

**Explanation** — this is the head-to-head summary. The arrows indicate which direction is better: SSIM and PSNR higher is better (closer to the true map), FID lower is better (generated and real maps look more alike). Pix2Pix leads on all three quality metrics, confirming the visual impression in Figure 3. The two generators are the same size at inference, but CycleGAN’s total training footprint is far larger because four networks are optimised at once, and it also takes longer to train — the cost of working without paired data.

## 5. Conclusion

---

This work shows that GAN architectures can generate conditional synthetic images and perform non-trivial domain translation, but their usefulness for downstream tasks is bounded by data richness and training budget. In Task 1, an AC-GAN built from scratch produced class-conditioned fruit images (85.6% discriminator class-accuracy), yet a classifier trained only on synthetic data trailed the real-data classifier by 33 points (60.0% vs. 93.3%), and adding more synthetic data did not close the gap. In Task 2, the supervised Pix2Pix clearly outperformed the unsupervised CycleGAN on every fidelity metric, while CycleGAN remained valuable for settings without paired data. Together the experiments give a clear, quantitatively grounded comparison of supervised and unsupervised generative frameworks and a realistic picture of when synthetic data does — and does not — substitute for real data.